INTRODUCTION TO PYTHON
UNIT 4: SCIENTIFIC COMPUTING & NUMERICAL ARRAYS WITH NUMPY
Curriculum: Lovely Professional University | School of Electronics & Communication Engineering
Reference: Python Programming: Using Problem Solving Approach by Reema Thareja & Harris et al.

>> 4.1 Foundations of NumPy and the ndarray Architecture

NumPy (Numerical Python) is the foundational scientific library powering Python's engineering ecosystem:
* Limitations of Standard Python Lists: Dynamic type pointers stored across fragmented memory locations; element-by-element loops incur massive interpreter dispatch overhead.
* The ndarray Object: Homogeneous, fixed-size multidimensional array stored in contiguous C-order or Fortran-order memory blocks. Supports Vectorization (pushing execution down to pre-compiled SIMD machine instructions).
* Core Attributes: arr.shape (tuple of dimensions), arr.ndim (number of axes), arr.dtype (data type, e.g. int32, float64, complex128), arr.size (total element count), arr.itemsize (bytes per element).
* Array Creation Functions: np.array(list), np.zeros(shape), np.ones(shape), np.full(shape, val), np.eye(N) (identity matrix), np.arange(start, stop, step), np.linspace(start, stop, num) (uniform linearly spaced intervals).

>> 4.2 Array Indexing, Slicing and Boolean Masking

* Multi-Dimensional Indexing: arr[row, col] (faster than arr[row][col] which creates intermediate list objects).
* Slicing: arr[row_start:row_stop, col_start:col_stop]. Slices return Views (shallow references sharing underlying memory buffer), not copies! Modifying a slice mutates the original array.
* Fancy Indexing: Passing arrays of integer indices to select elements in arbitrary non-contiguous orders.
* Boolean Masking: Applying relational expressions on arrays generates a boolean mask: mask = (arr > threshold). Indexing with mask (arr[mask]) extracts elements satisfying criteria in a single vectorized sweep.

>> 4.3 Vectorized Mathematics and Universal Functions (ufuncs)

* Universal Functions: Functions operating element-wise on ndarrays: np.sin(), np.cos(), np.exp(), np.log(), np.sqrt(), np.abs().
* Aggregation Functions: np.sum(arr, axis), np.mean(), np.std(), np.var(), np.min(), np.max(), np.argmin(), np.argmax().

* Broadcasting Semantics: Automatic dimension expansion enabling element-wise arithmetic between arrays of different shapes without memory copying. Rule: Two dimensions are compatible if they are equal or one of them is 1.

>> 4.4 Linear Algebra Module (numpy.linalg)

* Matrix Multiplication: np.dot(A, B) or matrix operator A @ B.
* Core Matrix Operations: np.linalg.det(A) (computes determinant), np.linalg.inv(A) (matrix inversion), np.linalg.matrix_rank(A), np.transpose(A) or A.T.
* Linear System Solver: np.linalg.solve(A, B) solves the matrix linear system A x = B via LU/Cholesky decomposition.
* Spectral Analysis: np.linalg.eig(A) computes eigenvalues and eigenvectors.

[PROGRAM 1: Vectorized Sinusoidal Signal Generation and Sampling]

Problem Context: Generate and analyze an analog sinusoidal signal v(t) = V_m * sin(2*pi*f*t + phi) sampled at Nyquist rate using NumPy vectorization.
Python Source Code:

```
import numpy as np

# Signal Specifications
f_signal = 1000.0 # 1 kHz frequency
v_peak = 5.0 # 5 Volts amplitude
phase_deg = 30.0 # 30 degrees initial phase

# Sampling Specifications (f_s >= 2 * f_signal)
f_sampling = 20000.0 # 20 kHz sampling frequency (20 samples per cycle)
duration = 0.002 # 2 milliseconds (2 full cycles)

# Generate time vector using np.linspace
total_samples = int(duration * f_sampling)
t = np.linspace(0, duration, total_samples, endpoint=False)

# Vectorized Signal Synthesis (zero explicit loops)
phi_rad = np.radians(phase_deg)
v_signal = v_peak * np.sin(2 * np.pi * f_signal * t + phi_rad)

# Statistical Array Aggregations
v_max = np.max(v_signal)
v_min = np.min(v_signal)
v_rms = np.sqrt(np.mean(v_signal**2))
v_avg = np.mean(np.abs(v_signal))
```

```
print(f'Total Samples Generated : {total_samples}')
print(f'Theoretical Peak : {v_peak:.2f} V | Measured Max: {v_max:.2f} V')
print(f'Theoretical RMS (Vp/r2) : {v_peak / np.sqrt(2):.4f} V | Measured RMS:
{v_rms:.4f} V')
print(f'Measured Average Value : {v_avg:.4f} V')
```

[PROGRAM 2: Multi-Mesh Electrical Circuit Solver via np.linalg.solve]

Problem Context: Solve the loop currents I1, I2, I3 for a 3-loop bridge circuit using matrix linear algebra: [R] * [I] = [V].

Python Source Code:

```
import numpy as np

# Resistance Matrix [R] in Ohms
R = np.array([
[ 12.0, -4.0, -2.0],
[ -4.0, 16.0, -6.0],
[ -2.0, -6.0, 14.0]
])

# Voltage Vector [V] in Volts
V = np.array([24.0, 0.0, -12.0])

# Verify non-singular matrix (det != 0)
det_R = np.linalg.det(R)
print(f'Determinant of R Matrix: {det_R:.2f}')

if abs(det_R) > 1e-6:
# Solve linear system R * I = V
I = np.linalg.solve(R, V)
print('Calculated Mesh Currents:')
for i, current in enumerate(I, start=1):
print(f' Loop Current I{i} = {current:8.4f} Amperes')

# Verify solution: R @ I must equal V
v_verify = R @ I
print(f'Residual Check (R @ I - V): {np.max(np.abs(v_verify - V)):.2e}')
else:
print('Singular matrix! Circuit has no unique solution.')
```

[PROGRAM 3: Digital Image Matrix Brightness & Inversion Processing]

Problem Context: Simulate grayscale digital image processing operations (brightness scaling, thresholding, negative inversion) on a 2D NumPy array.
Python Source Code:

```
import numpy as np

# Synthetic 4x4 grayscale image patch (values 0-255 uint8)
np.random.seed(42)
img = np.random.randint(50, 200, size=(4, 4), dtype=np.uint8)
print('Original Image Matrix:\n', img)

# 1. Negative Image Inversion: I_inv = 255 - I
img_inverted = 255 - img
print('\nInverted (Negative) Matrix:\n', img_inverted)

# 2. Brightness Adjustment (Vectorized clipping to prevent overflow)
brightness_boost = 40
img_bright = np.clip(img.astype(np.int16) + brightness_boost, 0, 255).astype(np.uint8)
print('\nBrightness Boosted (+40):\n', img_bright)

# 3. Binary Thresholding (Binarization at threshold = 128)
binary_mask = (img >= 128).astype(np.uint8) * 255
print('\nBinarized Black/White Matrix:\n', binary_mask)
```

[PROGRAM 4: Statistical Outlier Rejection via Boolean Masking]

Problem Context: Clean experimental RF signal power measurements by automatically identifying and rejecting statistical outliers beyond 2 standard deviations.
Python Source Code:

```
import numpy as np

# Experimental telemetry with spurious transient noise spikes
telemetry = np.array([45.2, 44.8, 45.0, 98.6, 44.9, 45.3, -12.4, 45.1, 44.7, 45.5])

mean_raw = np.mean(telemetry)
std_raw = np.std(telemetry)

# Vectorized Boolean Mask: keep samples within [mean - 2*std, mean + 2*std]
mask = (telemetry >= mean_raw - 2 * std_raw) & (telemetry <= mean_raw + 2 * std_raw)
clean_telemetry = telemetry[mask]
rejected_outliers = telemetry[~mask]

print(f'Raw Dataset Size : {len(telemetry)} | Raw Mean: {mean_raw:.2f}')
print(f'Rejected Outliers: {rejected_outliers}')
print(f'Clean Dataset : {clean_telemetry}')
```

```
print(f'Calibrated Mean : {np.mean(clean_telemetry):.4f} +/-
{np.std(clean_telemetry):.4f}')
```